

Reviewer Pack — Minimal Rank of Kähler Manifolds over Function Fields vs Qua...

Entry e7e00c9becc0 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-24 17:46:50 UTC

Minimal Rank of Kähler Manifolds over Function Fields vs Quantum Query Complexity for Bell's Theorem

Entry ID: e7e00c9becc0 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-24 17:46:50 UTC

1. Conjecture

Field A (mathematical branch): Kähler Geometry over Function Fields **Field B** (complexity object): Quantum Computing (Bell's Theorem Query Complexity)

Statement:

{'s1': "For every function field K , there exists a positive constant $c(K)$ such that the quantum query complexity for Bell's theorem is lower-bounded by the minimal rank of a suitably constructed Kähler manifold over K .", 's2': "Specifically, if M is a Kähler manifold with rank $r(M)$ and K is its function field, then $Q(\text{Bell}, n) \geq c(K) \cdot r(M)$, where $Q(\text{Bell}, n)$ is the quantum query complexity for Bell's theorem with n qubits.", 's3': 'For any instance with $n \leq 40$ qubits, if there exists a Kähler manifold M over K such that $r(M) < c(K)/n$, then $Q(\text{Bell}, n)$ cannot be lower-bounded by the minimal rank of M .'}

Rationale (proposer's reasoning):

{'s1': 'Kähler manifolds encode geometric structures that can potentially lead to non-trivial quantum phenomena, and their ranks might reflect the complexity of quantum information processing.', 's2': "Bell's theorem is a cornerstone in quantum mechanics that relates classical probabilities to quantum correlations, which could be linked to the geometric properties of Kähler manifolds.", 's3': 'This conjecture aims to explore an unexpected connection between geometry and quantum computing by leveraging the tools from complex analysis and algebraic geometry.'}

Taxonomy category: complexity_theory (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): d84bdac77840dd5f

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if no seed with $n \leq 40$ qubits generates a Kähler manifold with a rank less than $c(K)/n$ for all function fields K , where $Q(\text{Bell}, n) \geq c(K)$. The conjecture

is falsified if there exists at least one seed with $n \leq 40$ qubits that produces a Kähler manifold with rank $< c(K)/n$ for any function field K .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.80	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Kähler geometry over function fields" AND "Bell's theorem query complexity" - "quantum query complexity" AND minimal rank Kähler manifolds function field" - "quantum computing Bell's theorem" AND lower bound minimal rank Kähler manifold

Top relevant hits considered: - [http://arxiv.org/abs/1109.4165v2] Quantum Query Complexity of Subgraph Containment with Constant-sized Certificates - [http://arxiv.org/abs/0908.2468v1] Average/Worst-Case Gap of Quantum Query Complexities by On-Set Size - [http://arxiv.org/abs/2109.06917v1] Open Problems Related to Quantum Query Complexity

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A):
        m, n = len(A), len(A[0])
        for i in range(m):
            max_row = i
            for j in range(i+1, m):
                if abs(A[j][i]) > abs(A[max_row][i]):
                    max_row = j
            A[i], A[max_row] = A[max_row], A[i]
            pivot = A[i][i]
            for j in range(n):
                A[i][j] /= pivot
            for j in range(m):
                if j != i:
                    factor = A[j][i]
                    for k in range(n):
                        A[j][k] -= factor * A[i][k]
```

```

    return A

def matrix_multiplication(A, B):
    m, n, p = len(A), len(B), len(B[0])
    C = [[0]*p for _ in range(m)]
    for i in range(m):
        for j in range(p):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C

def is_invertible(A):
    m, n = len(A), len(A[0])
    if m != n:
        return False
    det = 1
    for i in range(m):
        max_row = i
        for j in range(i+1, m):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]
        det *= A[i][i]
    return det != 0

def rank_of_matrix(A):
    rref = gaussian_elimination(A)
    rank = sum(1 for row in rref if any(row[j] != 0 for j in range(len(row))))
    return rank

n = random.choice([5, 10, 15, 20, 30, 40])
K = [random.randint(-10, 10) for _ in range(n)]
M = [[random.randint(0, 1) for _ in range(n)] for _ in range(n)]

if not is_invertible(M):
    return {
        "metric_name": "rank_of_matrix",
        "metric_value": None,
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": "Matrix M is not invertible"
    }

rank_M = rank_of_matrix(M)
c_K = sum(abs(k) for k in K)

if rank_M < c_K / n:
    return {
        "metric_name": "rank_of_matrix",
        "metric_value": rank_M,
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": f"Rank of M ({rank_M}) is less than c(K)/n ({c_K/n})"
    }

return {

```

```

        "metric_name": "rank_of_matrix",
        "metric_value": rank_M,
        "instances_tested": 1,
        "conjecture_holds": True,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_value = sum(result["metric_value"] for result in results if result["metric_value"] is not None)
    std_value = math.sqrt(sum((result["metric_value"] - mean_value)**2 for result in results if result["metric_value"] is not None) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next((seed for seed, result in zip(seeds, results) if not result["conjecture_holds"]))
        print(f"RESULT: FALSIFIED counterexample=\"Rank of M is less than c(K)/n\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

ix M is not invertible'}
TRIAL: {'metric_name': 'rank_of_matrix', 'metric_value': None, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'rank_of_matrix', 'metric_value': 15, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'rank_of_matrix', 'metric_value': None, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'rank_of_matrix', 'metric_value': None, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'rank_of_matrix', 'metric_value': None, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'rank_of_matrix', 'metric_value': None, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'rank_of_matrix', 'metric_value': None, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'rank_of_matrix', 'metric_value': None, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b4ca3b3e.py", line 106, in <module>
    trial_result = run_trial(seed)
                   ~~~~~^~~~~~

  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b4ca3b3e.py", line 80, in run_trial
    rank_M = rank_of_matrix(M)
             ~~~~~^~~~~~

  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b4ca3b3e.py", line 63, in rank_of_matrix
    rref = gaussian_elimination(A)
           ~~~~~^~~~~~

```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b4ca3b3e.py", line 31, in gaussian_elim
    A[i][j] /= pivot
ZeroDivisionError: float division by zero
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed due to a division by zero error before producing data that would allow for a definitive conclusion on the conjecture. | next: Investigate and fix the division by zero error in the code, then rerun the test with seeds up to $n \leq 40$ qubits to determine if the conjecture is supported or falsified.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	17149
2	preregistration	ollama_remote	glm4:latest	0	0	6094
3	novelty	ollama_remote	glm4:latest	0	0	4711
4	novelty	ollama_remote	glm4:latest	0	0	6458
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	32164
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	6470
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7219
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11957
9	judge	ollama_remote	glm4:latest	0	0	8780

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 101002 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/e7e00c9becc0.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/e7e00c9becc0.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/e7e00c9becc0.tar.gz (if generated)